

Módulo 15: Arquitectura Hexagonal (Puertos y Adaptadores)

Contenidos clave

Arquitectura Hexagonal: La lógica de negocio no debe depender de tecnologías externas.

Capas:

- **dominio:** en esta capa viven las reglas de negocio, su objetivo es representar el negocio.
- **aplicación:** en esta capa están los casos de uso. Un caso de uso orquesta las operaciones necesarias; pero no necesariamente sabe cómo se guarda o cómo se envía la notificación.
- **infraestructura:** en esta capa viven los detalles tecnológicos: SQLAlchemy, PostgreSQL, FastAPI, HTTPX, Kafka, Redis, etc..

Puertos (interfaces/Protocols) y adaptadores (SQL, HTTP, mensajería):

- Puertos: un puerto es un contrato que define lo que se necesita, pero no cómo se hace. En Python se puede usar Protocol.
- Adaptadores: un adaptador implementa un puerto utilizando una tecnología concreta.

Casos de uso y orquestación: un caso de uso representa una acción que el sistema puede realizar desde el punto de vista del negocio.

DTOs vs entidades: DTO sirve para transportar datos, su función es representar los datos que entran o salen de la aplicación. Entidad representa un concepto del dominio, puede contener comportamiento y reglas de negocio, una acción completa del negocio ("Crear una orden para un cliente y notificar que fue creada"). Orquestar significa coordinar diferentes componentes para completar un caso de uso.

Inyección de dependencias y wiring en FastAPI: Wiring significa conectar las piezas. En FastAPI normalmente se hace mediante Depends(), así FastAPI se encarga de proporcionar las dependencias.

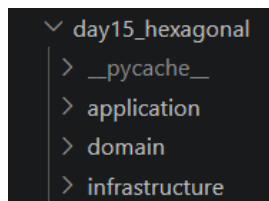
Pruebas de dominio, contrato y end-to-end:

- Pruebas de dominio: comprueban que las reglas de negocio funcionan correctamente. Se enfocan en entidades y lógica del dominio, sin importar SQLAlchemy, HTTP, FastAPI, etc.. Son pruebas rápidas y aisladas.
- Pruebas de contrato: comprueban que un adaptador cumple el contrato definido por un puerto. Verifica que ambos se comportan como el puerto espera.

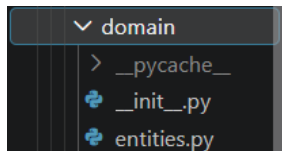
Objetivos

- **Separar reglas de negocio de detalles de infraestructura.**

La separación se implementó mediante 3 capas principales:



- Capa de Dominio: Contiene definidas las entidades y reglas de negocio.



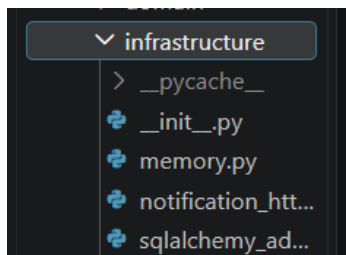
La lógica del subtotal pertenece al dominio y no depende de: SQLAlchemy, HTTP, FastAPI, bases de datos ni servicios externos. La orden calcula su total igual sin importar si la orden viene de memoria, SQLAlchemy o una API externa.

- Capa de Aplicación: En esta capa se implementó el caso de uso CreateOrder, recibe abstracciones, no clases concretas, la lógica de creación:

```
15     def execute(  
16         self,  
17         data: CreateOrderInput,  
18     ) -> OrderOutput:  
19         if not data.items:  
20             raise ValueError("La orden debe contener al  
21  
22         items = tuple(  
23             OrderItem(  
24                 product_name=item.product_name,  
25                 quantity=item.quantity,  
26                 unit_price=item.unit_price,  
27             )  
28             for item in data.items  
29         )  
30  
31         order = Order(  
32             id=self._repository.next_id(),  
33             customer_id=data.customer_id,  
34             items=items,  
35         )  
36  
37         saved_order = self._repository.save(order)  
38         self._notification.order_created(saved_order)  
39  
40         return OrderOutput(  
41             order_id=saved_order.id,  
42             customer_id=saved_order.customer_id,  
43             total=saved_order.total,  
44             status=saved_order.status,  
45         )
```

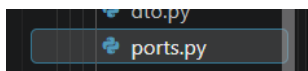
Contiene la orquestación del caso de uso, pero no sabe cómo se genera el ID internamente, dónde se guarda la orden ni cómo se envía la notificación.

- Capa de Infraestructura: Aquí se implementaron los detalles concretos.



Contiene el repositorio en memoria, el repositorio en SQLAlchemy y el notificador HTTP. La infraestructura depende de los contratos definidos por la aplicación, no al contrario.

- Definir puertos estables y adaptadores intercambiables.



Se definió el puerto del repositorio:

```
6 class OrderRepository(Protocol):
7     def next_id(self) -> int: ...
8
9     def save(self, order: Order) -> Order: ...
10
11     def get_by_id(
12         self,
13         order_id: int,
14     ) -> Order | None: ...
15
```

Y el puerto de notificaciones:

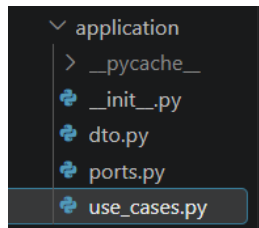
```
17 class OrderNotification(Protocol):
18     def order_created(
19         self,
20         order: Order,
21     ) -> None: ...
22
```

Estos protocolos son contratos estables. El caso de usos solo necesita saber que el repositorio puede generar un ID, guardar una orden y consultar una orden; y que la notificación puede informar que una orden fue creada, no necesita conocer la tecnología que hay detrás.

Laboratorio

- Caso de uso CreateOrder con puerto de repositorio y adaptadores en memoria/SQLAlchemy.

Caso de uso implementado:



Tiene la responsabilidad de:

- Validar que la orden tenga elementos
- Convertir DTOs en entidades de dominio
- Solicitar un nuevo ID al repositorio.
- Crear la entidad Order
- Guardarla mediante el puerto
- Notificar su creación
- Devolver un DTO de salida

Con el uso de DTOs se evita exponer directamente las entidades internas.

- Adaptador de repositorio en memoria.

```
class InMemoryOrderRepository(OrderRepository):
    def __init__(self) -> None:
        self._orders: dict[int, Order] = {}
        self._sequence = 0

    def next_id(self) -> int:
        self._sequence += 1
        return self._sequence

    def save(self, order: Order) -> Order:
        self._orders[order.id] = order
        return order

    def get_by_id(
        self,
        order_id: int,
    ) -> Order | None:
        return self._orders.get(order_id)
```

Este adaptador se utiliza para: pruebas rápidas, desarrollo sin base de datos y verificación del caso de uso de forma aislada. También se implementó un notificador en memoria que permite verificar que la notificación se ejecutó:

```
25 class InMemoryOrderNotification(OrderNotification):
26     def __init__(self) -> None:
27         self.created_orders: list[Order] = []
28
29     def order_created(self, order: Order) -> None:
30         self.created_orders.append(order)
31
```

- Adaptador SQLAlchemy.

Su modelo de persistencia:

```
class OrderRow(Base):
    __tablename__ = "hexagonal_orders"

    id: Mapped[int] = mapped_column(
        primary_key=True,
    )

    customer_id: Mapped[int] = mapped_column(
        nullable=False,
    )

    total: Mapped[Decimal] = mapped_column(
        nullable=False,
    )

    status: Mapped[str] = mapped_column(
        nullable=False,
    )
```

Su implementación del repositorio:

```
class SQLAlchemyOrderRepository(OrderRepository):
    def __init__(
        self,
        session: Session,
    ) -> None:
        self._session = session
```

Generación de IDs:

```
def next_id(self) -> int:
    last_id = self._session.scalar(
        select(OrderRow.id).order_by(OrderRow.id.desc())
    )

    return (last_id or 0) + 1
```

Guardado:

```
def save(
    self,
    order: Order,
) -> Order:
    row = OrderRow(
        id=order.id,
        customer_id=order.customer_id,
        total=order.total,
        status=order.status,
    )

    self._session.merge(row)
    self._session.commit()

    return order
```

Consulta:

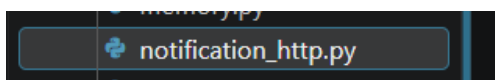
```
def get_by_id(
    self,
    order_id: int,
) -> Order | None:
    row = self._session.get(
        OrderRow,
        order_id,
    )

    if row is None:
        return None

    return Order(
        id=row.id,
        customer_id=row.customer_id,
        items=(),
        status=row.status,
    )
```

Este adaptador puede sustituirse por el adaptador en memoria sin modificar CreateOrder.

- Adaptador de notificación HTTP simulado.



- Implementación:

```
8 @dataclass
9 class HttpOrderNotification:
10     endpoint: str
11     client: httpx.Client
```

- Método:

```
13 def order_created(self, order: Order) -> None:
14     response = self.client.post(
15         self.endpoint,
16         json={
17             "order_id": order.id,
18             "customer_id": order.customer_id,
19             "total": str(order.total),
20             "status": order.status,
21         },
22     )
23
24     response.raise_for_status()
25
```

El adaptador traduce la entidad de dominio al formato esperado por el proveedor HTTP. El caso de uso no conoce http.Client ni client.post(...) , solo conoce notification.order_created(order).

- Pruebas de contrato

- Pruebas de dominio y aplicación:

Se probó la creación de una orden:

```
def test_create_order():
    repository = InMemoryOrderRepository()
    notification = InMemoryOrderNotification()
    use_case = CreateOrder(repository, notification)

    result = use_case.execute(build_input())

    assert result.order_id == 1
    assert result.customer_id == 10
    assert result.total == Decimal("100.00")
    assert result.status == "PENDING"
    assert len(notification.created_orders) == 1
```

Verificando la creación de la orden, el cálculo correcto del total, su estado inicial, persistencia y notificación. Y su validación:

```
def test_create_order_requires_items():
    repository = InMemoryOrderRepository()
    notification = InMemoryOrderNotification()
    use_case = CreateOrder(repository, notification)

    data = CreateOrderInput(
        customer_id=10,
        items=(),
    )

    with pytest.raises(
        ValueError,
        match="al menos un elemento",
    ):
        use_case.execute(data)
```

- Prueba de contrato SQLAlchemy:

Se probó el mismo caso de uso con el repositorio SQLAlchemy:

```
# Pruebas de contrato
def test_sqlalchemy_repository_contract():
    engine = create_sqlite_engine()
    Base.metadata.create_all(bind=engine)

    with Session(engine) as session:
        repository = SQLAlchemyOrderRepository(session)
        notification = InMemoryOrderNotification()
        use_case = CreateOrder(repository, notification)

        result = use_case.execute(build_input())

        assert result.order_id == 1
        assert result.customer_id == 10
        assert result.total == Decimal("100.00")
        assert result.status == "PENDING"

    engine.dispose()
```

Comprobando que el adaptador SQLAlchemy cumple con el contrato requerido por CreateOrder.